

MCPPRO Core Architecture Specification

MCPPRO Core Architecture Specification

Version: 2.4.0

Author: Systems Engineering Team

1. Executive Summary

MCPPRO is a distributed intelligence orchestration engine combining Model Context Protocol (MCP) server integration, Document Retrieval-Augmented Generation (RAG), and dynamic autonomous agent loops. The primary objective is to allow AI agents to securely query unstructured documents, execute remote tools via standard JSON-RPC over HTTP/Stdio transports, and synthesize verified answers with zero hallucination.

2. Component Layers

2.1 Client Layer

The client layer is implemented in Next.js 14 using React Server Components, Tailwind CSS, and Radix UI primitives. It interfaces with the backend through both streaming chat endpoints and modular REST execution routes.

2.2 Frontend Orchestration Engine

Located in `/frontend/app/api/mcppro-agent/run/route.ts`, the frontend orchestrator handles query sanitization, loads static tools (JavaScript execution, code file creation, Supabase queries), and dynamically polls active MCP tool registries using `MCPClientManager`. The autonomous agent loop runs up to 15 iterative reasoning steps using Vercel AI SDK.

2.3 Backend RAG Engine

The backend is built with Python 3.12+ and FastAPI. It exposes high-throughput endpoints for vector indexing and question answering. It features two execution modes: Agentic Mode (governed by `MasterMCPro`) and Traditional RAG (one-shot cosine retrieval via `LangChain`).

3. Technical Parameters

- Default Vector Store: InMemory / Qdrant
- Default Embedding Dimension: 1536 (OpenAI text-embedding-3-small) / 1024 (BGE-M3)
- Chunking Boundary: 1000 characters with 200 character overlap
- Maximum Agent Iterations: 15 steps
- Caching Key: SHA-256 digest of document canonical URL